Task Description #1 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file.

Analyze: Check if the AI stores sensitive data in plain text or without encryption.

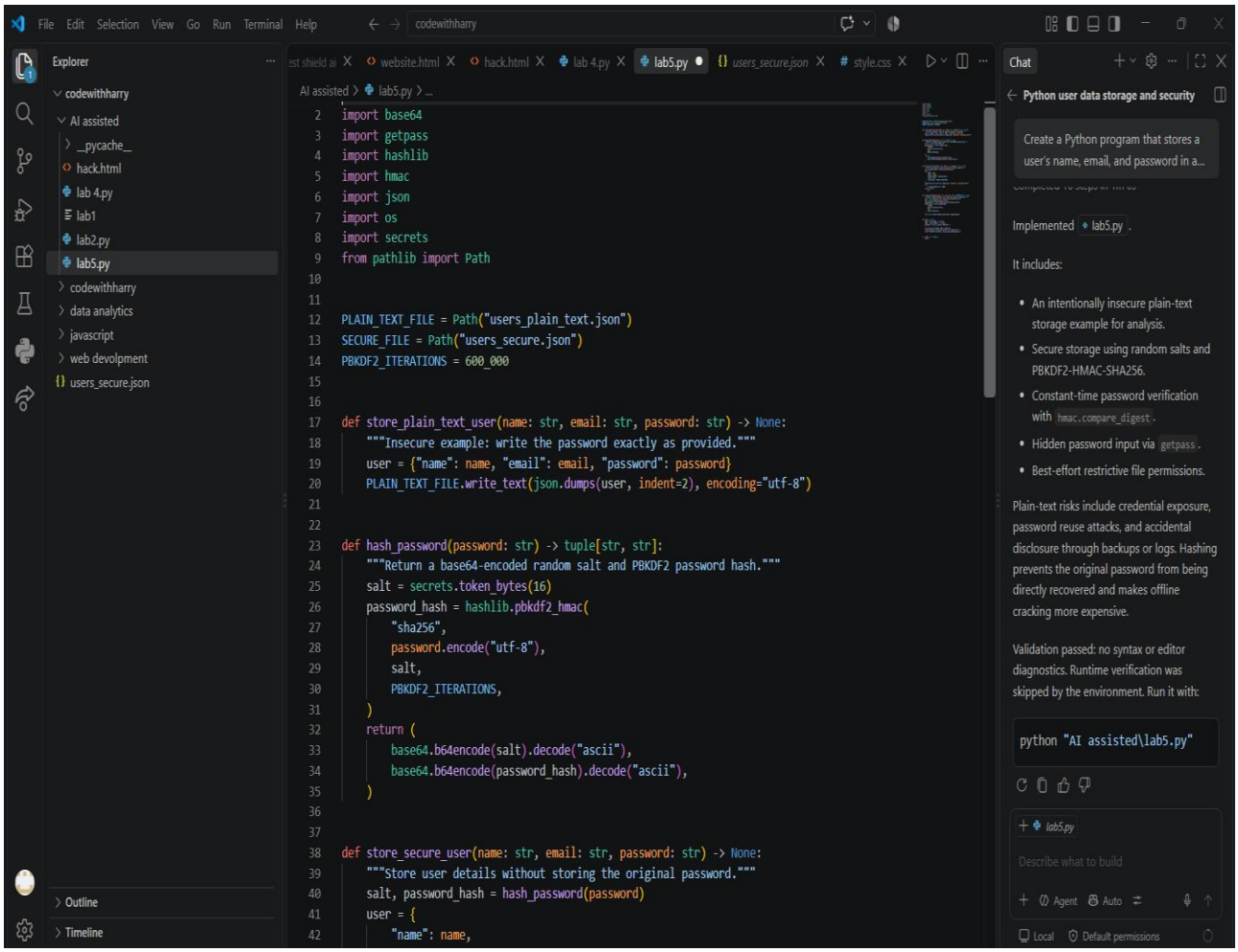
Expected Output:

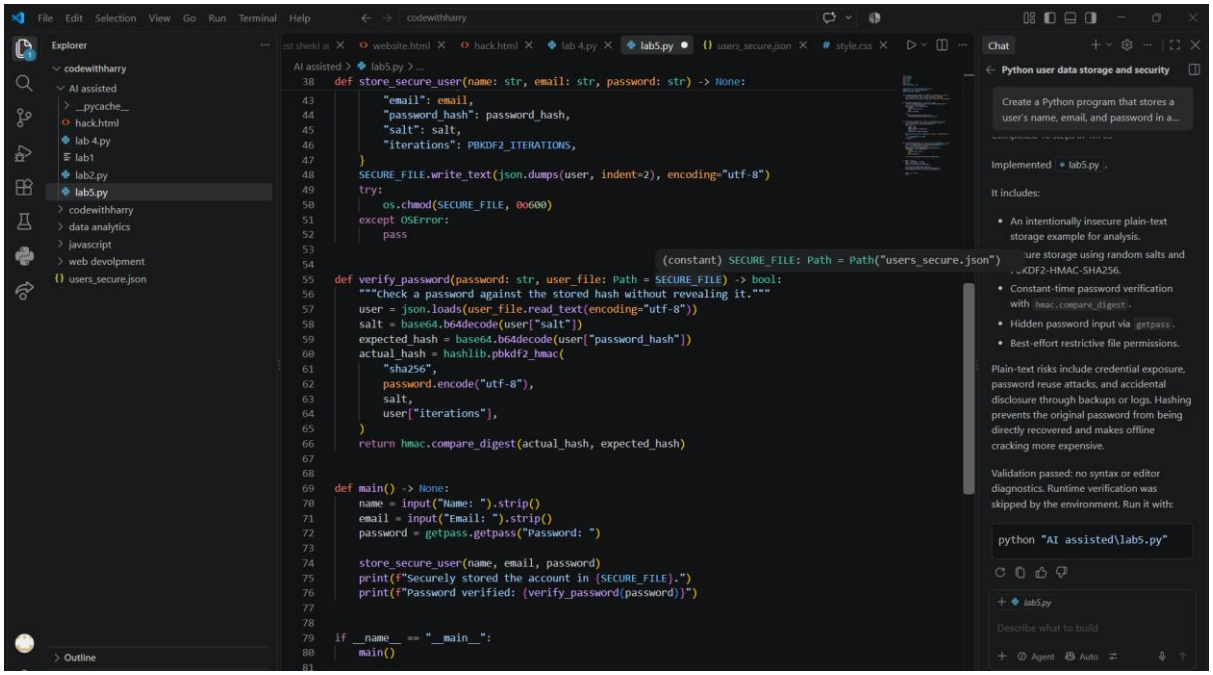
Identified privacy risks.

Revised version with encrypted password storage (e.g., hashing).

PROMPT:- Create a Python program that stores a user's name, email, and password in a file. Analyze the privacy and security risks of storing the password in plain text, then provide a revised version that securely stores the password using hashing and explain the improvements.

Code:-





```python
def store_secure_user(name: str, email: str, password: str) -> None:
        "email": email,
        "password_hash": password_hash,
        "salt": salt,
        "iterations": PBKDF2_ITERATIONS,
    }
    SECURE_FILE.write_text(json.dumps(user, indent=2), encoding="utf-8")
    try:
        os.chmod(SECURE_FILE, 0o600)
    except OSError:
        pass

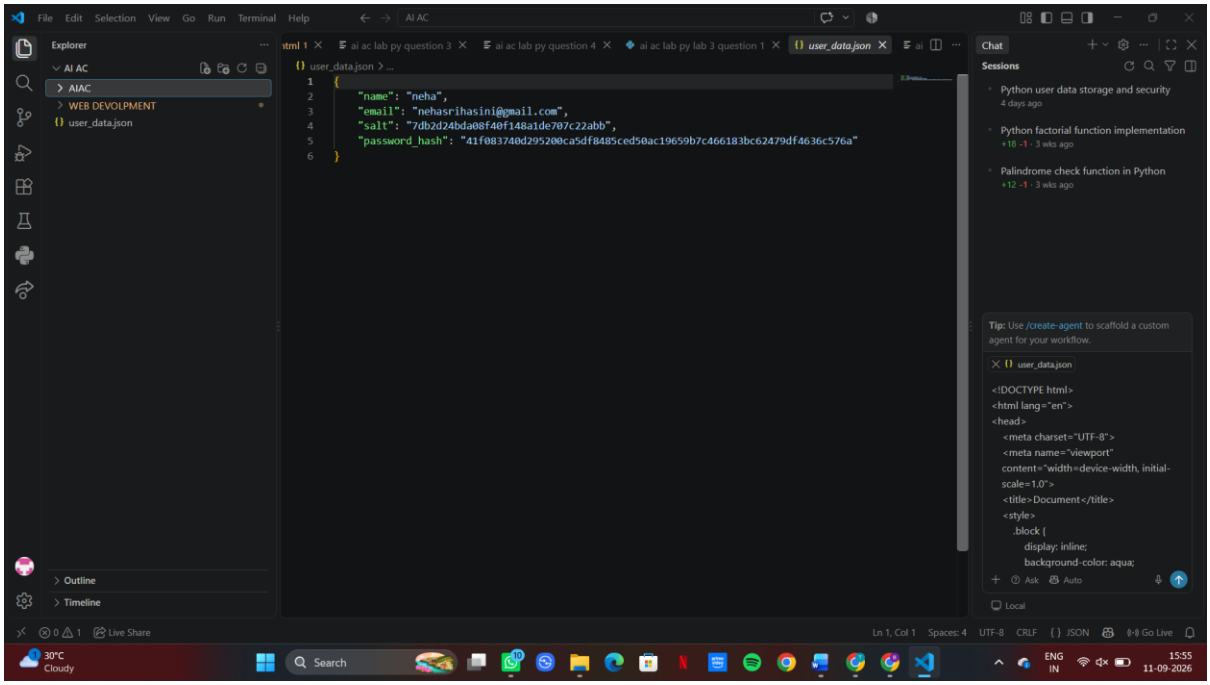
def verify_password(password: str, user_file: Path = SECURE_FILE) -> bool:
    """Check a password against the stored hash without revealing it."""
    user = json.loads(user_file.read_text(encoding="utf-8"))
    salt = base64.b64decode(user["salt"])
    expected_hash = base64.b64decode(user["password_hash"])
    actual_hash = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        user["iterations"],
    )
    return hmac.compare_digest(actual_hash, expected_hash)


def main() -> None:
    name = input("Name: ").strip()
    email = input("Email: ").strip()
    password = getpass.getpass("Password: ")

    store_secure_user(name, email, password)
    print(f"Securely stored the account in {SECURE_FILE}.")
    print(f"Password verified: {verify_password(password)}")


if __name__ == "__main__":
    main()
```

RESULT:-



```
{
    "name": "neha",
    "email": "nehasrihasini@gmail.com",
    "salt": "7db2d24bda08f40f148a1de707c22abb",
    "password_hash": "41f083740d295200ca5df8485ced50ac19659b7c466183bc62479df4636c576a"
}
```

Task Description #2 – Fair Decision Logic

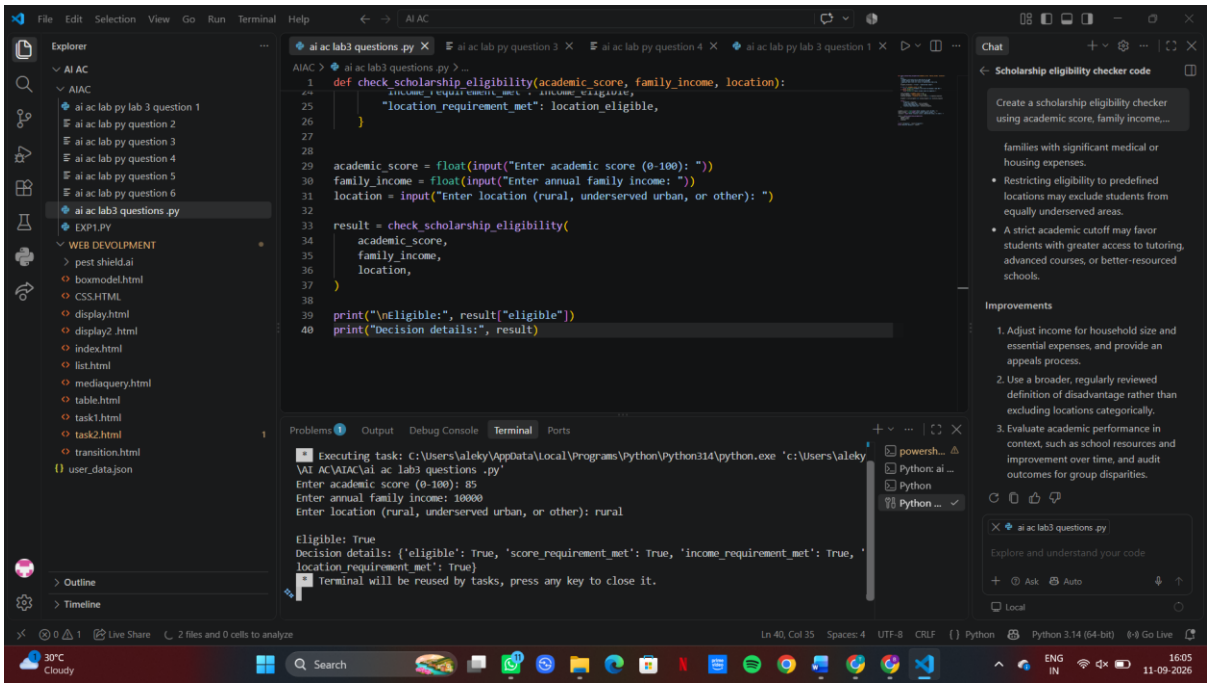
Task: Prompt an AI tool to generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

Expected Output-2:

Code snippet with observations on fairness. Brief explanation (3–4 sentences) suggesting improvements to ensure equitable decision-making**.**

Prompt:-

Create a scholarship eligibility checker using academic score, family income, and location. Generate the code and identify any unfair bias in the decision logic, then suggest 2–3 improvements for fair and equitable decisions.



Task Description #3 (Transparency in AI-Based Admission System)

Task: Use an AI tool to generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

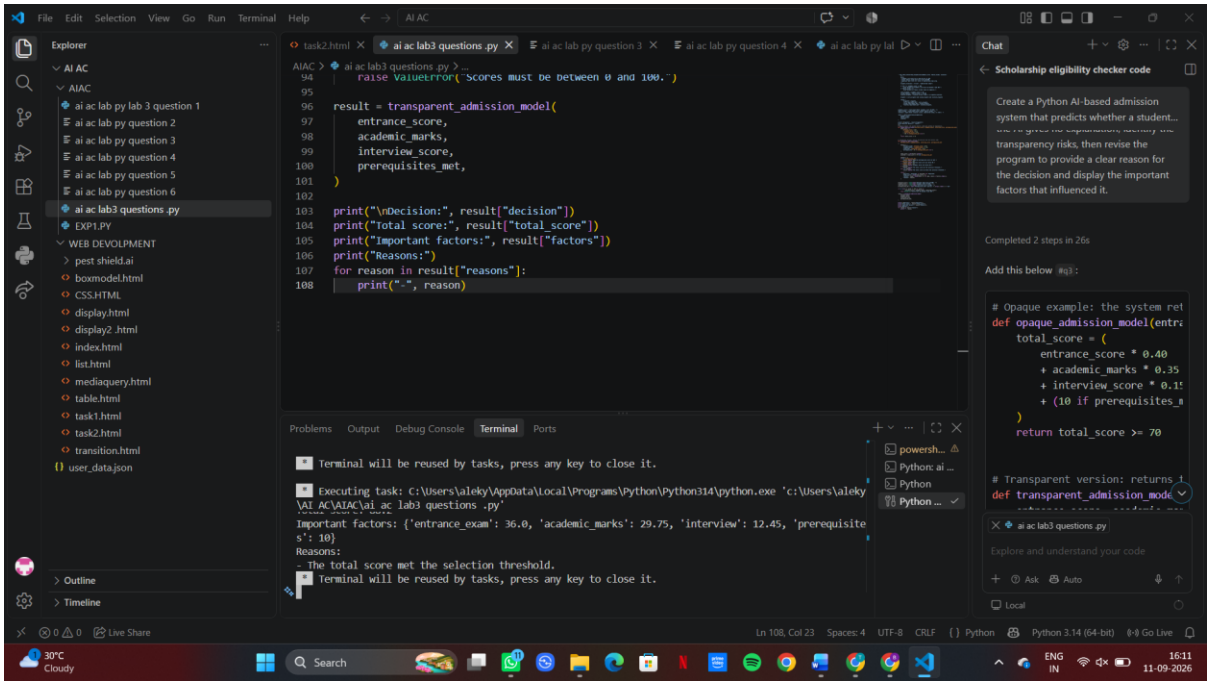
Analyze: Check whether the AI system provides a clear and understandable reason for its admission decision.

Expected Output:

- Identified transparency risks.
- Example of an admission decision where the AI gives no explanation.
- Revised version that provides a clear reason for selecting or rejecting the student.
- Example output showing the important factors that influenced the AI's decision.

Prompt: Create a Python AI-based admission system that predicts whether a student is selected or rejected using entrance exam score, academic marks, and other relevant criteria. Show an example where the AI gives no explanation, identify the transparency

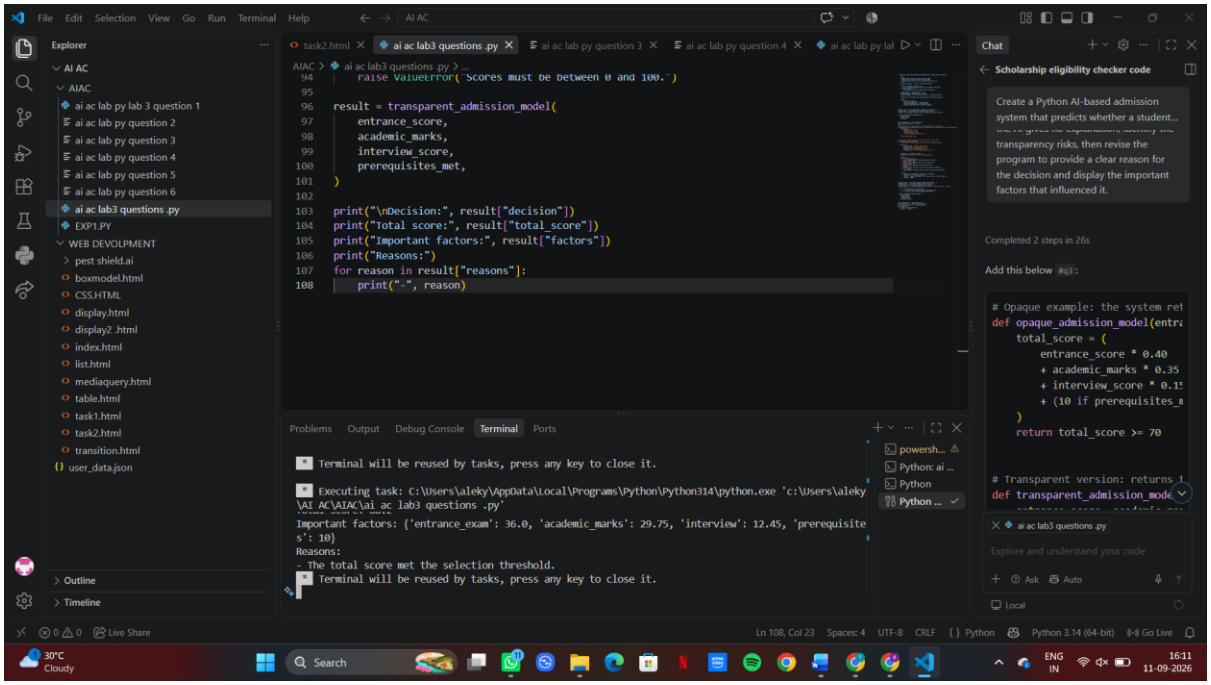
risks, then revise the program to provide a clear reason for the decision and display the important factors that influenc



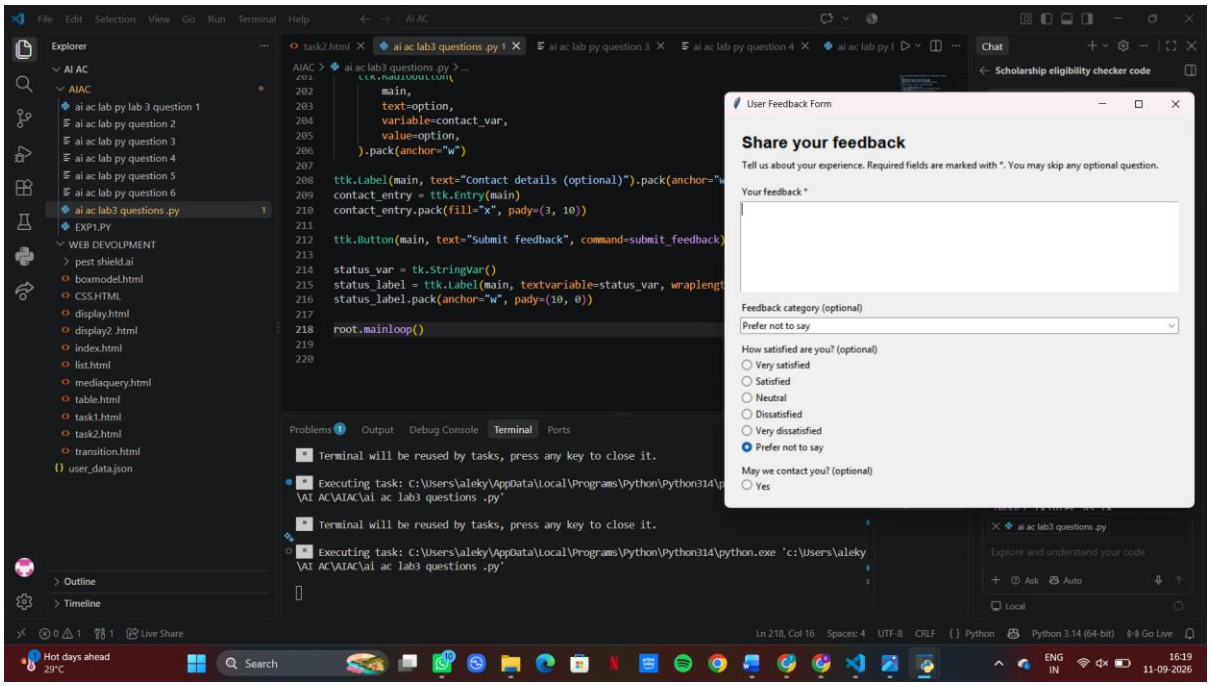
Task Description #4 – Ethical Scoring System

> **Task:** Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.
>
> **Expected Output-4:**
>
> Python code along with a written analysis identifying whether the evaluation criteria are balanced and justifiable.

Prompt: Create a Python employee performance evaluation system using project completion rate, teamwork score, and attendance. Analyze the scoring logic for unethical weighting or hidden bias, explain whether the criteria are balanced and justifiable, and suggest improvements for a fairer evaluation system.



Prompt: Create a user feedback form application. Review the form for inclusive language, accessible design, and options that support diverse users without making exclusionary assumptions. Provide a revised version of the code with inclusive language, accessibility features, and flexible response options.



Task Description #6 (Fairness in Loan Approval System)

Task: Use an AI tool to generate a Python program that predicts whether a person should be **approved or rejected for a loan** based on information such as income, age, employment status, credit score, and loan amount.

Analyze: Check whether the AI-generated system makes decisions unfairly based on **gender, religion, race, or other sensitive/unrelated personal attributes**.

Expected Output:

- Identified fairness and discrimination risks.
- Examples showing how biased data or rules can lead to unfair loan decisions.
- Revised version that makes loan decisions based on **relevant financial factors** rather than sensitive attributes.
- Brief explanation of how the revised system improves fairness.

Prompt: Generate a Python loan approval system using income, age, employment status, credit score, and loan amount. Analyze whether it makes unfair decisions based on gender, religion, race, caste, or other sensitive attributes. Give examples of possible bias and then provide a revised version that uses only relevant financial factors and treats applicants fairly. Briefly explain how the revised system improves fairness.

